Isolating the Safety Function on a Single-Chip Wi-Fi SoC: Formal Verification, Response-Time Analysis, and a Reproducible Measurement Protocol

MD Humayun
Department of Computer Science and Engineering
Sardar Vallabhbhai National Institute of Technology, Surat, India

Abstract

Consumer intrusion alarms run almost entirely on single-chip Wi-Fi systems-on-chip, where the radio driver, the network stack and the safety application share one die, one scheduler and one interrupt controller. Engineers mitigate this by pinning the safety task to its own core and by gating risky features behind build configuration. Both are *assertions*: the first asserts a timing property, the second a security property, and neither is checked. This paper reports what happens when each is falsified cheaply.

We present an ESP32 intrusion alarm whose safety loop is pinned to the application core and separated from the network task by bounded, non-blocking queues and a zero-timeout status snapshot, with HMAC-SHA256 command authentication over a non-volatile monotonic counter. We specify the isolation argument in TLA+ and verify it exhaustively under a network task given no fairness assumption, and evaluate the system with 9,300 seeded adversarial trials and 3,000,000 sanitizer-backed fuzzing iterations.

Two inexpensive methods each found a defect that none of that detected. An ablation over eleven configurations found a compile-time guard accepting TLS plus a broker login as sufficient authority for unauthenticated remote disarm. The inference is unsound—the broker terminates TLS and re-originates every message—and under credential compromise the alarm fires in 16.7% of trials against 100% for an otherwise identical control. The principle is textbook and the attack class published, so we claim no new vulnerability; the point is that a guard encoding it compiles silently and costs a *safety* property. Separately, a response-time decomposition exposed an unbounded drain in the safety path that had survived every test, fuzz iteration, trial and model-checking run.

The ablation further shows that availability, integrity and the safety function fail independently, and that broker-side authorization and application-layer authentication fail under *different* compromises, so neither dominates: an access-control list contains a compromised device credential but not a compromised controller, and a message authentication code does the reverse. Only the layered configuration survives every case. All eight access-control scenarios were reproduced against a real Mosquitto broker, which agreed with the model throughout. Under key compromise the monotonic replay counter also locks the legitimate owner out, admitting 0 of 1250 commands—a negative result we did not anticipate.

These results establish *logical* isolation only. Physical timing invariance is a property of the silicon and a closed-source radio driver, and we report no latency figure we have not measured; in its place we supply the decomposition above, which leaves a single composite hardware quantity outstanding, and a public framework with which any laboratory holding two inexpensive boards can measure it.

Keywords: embedded systems security; real-time task isolation; formal verification; TLA+; IoT; ESP32

1 Introduction

A domestic intrusion alarm is a safety-critical system that is bought cheaply and then forgotten. Its single obligation is to produce a loud noise within a bounded time of an intrusion. If an attacker standing outside the building can delay or suppress that noise using inexpensive equipment, the device has failed at the only task it has.

Almost all such devices are now built on single-chip Wi-Fi systems-on-chip. On the ESP32, which dominates this market segment, the Wi-Fi driver, the TCP/IP stack, the MQTT client and the alarm application all execute on one die, under one FreeRTOS scheduler, sharing one interrupt controller. This is a materially different situation from the one studied in most of the real-time literature, where the attacked network interface is a separate device and the protected computation can be placed beyond its reach.

The mitigation practitioners reach for is core pinning: place the safety task on one core and everything network-facing on the other. The advice is reasonable and inexpensive, and it is what the platform makes available. Whether it is sufficient is a separate question, and a partially answered one. Behnke et al. [1] measured a periodic critical task's lateness on an ESP32 under IP packet load with the network server already pinned to the other core, and found measurable degradation: the task lost 2.15–3.8% of its cycles for packets addressed to a listened port and 3.17–6.67% for an unused port, with lateness growing roughly linearly once the microcontroller saturated. Their conclusion was that command-and-control over IP is unsuitable for hard real-time work on such devices.

That finding closes one question and opens three others, which motivate this paper.

First, their work measures the *phenomenon* but does not evaluate a *mitigation*. The follow-up work from the same group proposes two: a priority-aware multiqueue network interface with per-flow interrupt moderation [2], and driver-level early demultiplexing [3]. Neither is available to a developer shipping a stock ESP32. The first was evaluated against a simulated network interface and was never built; the second was prototyped on a Xilinx Zynq-7000, a far larger platform, with measurements taken on a single core. Critically, [3] states that the ESP32's second core is used to separate the observed processes from the test harness rather than as a mitigation, so core pinning is nowhere evaluated as a defence.

Second, all load in that line is applied at the IP layer, as UDP or MODBUS/TCP traffic. The attack that actually threatens a wireless alarm is deauthentication: 802.11 management frames that are processed in the MAC layer and the radio driver, beneath the IP stack, and that the device must handle whether or not any socket is listening. Whether this different interference path behaves like IP load is unknown.

Third, none of this work treats the isolation argument formally, and none models an adversary rather than volumetric stress.

1.1 Contributions

This paper's central claim follows from two such cases, found in our own system:

> A security property asserted in build configuration, and a timing property asserted in design, are both unverified claims. Each can be falsified by a method that costs far less than the system it checks.

Neither method we used is novel. What we report is that applying them to a system's *assumptions*, rather than to its running behaviour, caught two defects that testing, fuzzing, exhaustive model checking and 9,300 adversarial trials all missed. A third instance turned up in our own formal work: the invariant stating that unauthenticated remote disarm never takes effect was vacuous in the specification we shipped, because it *assumed* the refusal instead of deriving it—the same unsound inference the compile guard made (Section 6.3). We make the following contributions in support of that claim, each stated with the evidence behind it and the limits of that evidence. We claim no new mechanism, attack, protocol or formalism.

1. **A configuration-logic defect, found by ablation, that costs a safety property.** A compile-time guard permitted unauthenticated remote disarm whenever TLS and a broker login were configured. The inference is unsound: the broker terminates TLS and re-originates every message, so an arriving command carries no evidence of its publisher. Under credential compromise the alarm fires in 16.7% of trials against 100% for the otherwise identical control. The principle is textbook and the attack class is published [19], so we claim no new vulnerability. The contribution is that a guard encoding this inference compiles silently, emits no diagnostic, survives review, and costs a *safety* rather than merely a security property—and that an ablation with a fair baseline is sufficient to find it (Sections 9 and 11).

2. **A response-time decomposition that found a defect nothing else could.** Decomposing the safety loop's worst-case response time shows the blocking term is identically zero by construction, the loop's own work is constant-time, and the residual is a single composite hardware quantity—reducing four unknowns to one and identifying exactly what a measurement must isolate. In doing so it exposed an unbounded drain in the safety path that had survived 120 host checks, 3,000,000 sanitizer-backed fuzz iterations, 9,300 adversarial trials and exhaustive model checking (Section 5).

3. **An ablation separating three properties that are routinely conflated.** One adversary, one system, one seed set, eleven configurations. Availability, integrity and the safety function fail independently: a rate limiter costs 41% of operator commands while buying integrity nothing, a static token does the reverse, a transport-only configuration is a strong baseline exactly as long as broker credentials hold, and only a counter-bound MAC provides both. Sweeping the deauthentication detector over the same harness also corrected our own firmware default, which fired spuriously in 79.9% of benign ten-minute runs (Section 9).

4. **A compromise-indexed comparison of broker authorization against application-layer authentication, validated against a real broker.** Neither control dominates: a per-client ACL contains a compromised device credential and fails against a compromised controller; a MAC does the reverse; a shared credential collapses the ACL entirely; only the layered configuration survives every compromise tested. All eight scenarios were reproduced against Mosquitto 2.0.18 with TLS and per-client ACLs, which agreed with the model in every case. The comparison also yields a negative result we did not anticipate: under key compromise the monotonic replay counter locks the legitimate owner out of the system entirely (Sections 9.3 and 9.3.1).

5. **A reproducible artifact with an explicit evidence taxonomy.** Firmware, specifications, harnesses, analysis scripts and figure sources are public, and results regenerate byte-identically from committed seeds. Every number is labelled by evidence category, no physical quantity is reported that was not measured, and we document four harness defects that produced plausible but wrong numbers—two of which flattered the conclusion the experiment was built to support.

The isolation design itself (core pinning, bounded non-blocking queues, a zero-timeout status snapshot, and the TLA+ specification of the resulting argument, verified under a network task given no fairness assumption and mutation-tested as described in Section 6.3) is described in Sections 4 and 6. We present it as the system under study rather than as a contribution: each element is standard real-time practice, and prior work has already measured task lateness on this SoC under on-chip network load [1].

1.2 Scope and honesty of claims

This paper establishes *logical* isolation: no control- or data-flow dependency allows network activity to block, delay or corrupt the safety path, and no forged, stale or malformed command is accepted. It does *not* establish *timing* isolation, which concerns contention for shared silicon and is a property of the implementation rather than the design. Given [1], we regard it as likely that some timing interference exists; the open question is its magnitude relative to the deadline margin, and whether queue discipline

bounds it. We state this distinction wherever a result could be read more strongly than the evidence supports.

2 Background and Related Work

2.1 Real-time degradation from on-chip network activity

The closest work to ours is the line from Behnke and colleagues, summarised in Table 1. Their 2020 study [1] is the measurement that motivates this paper, and is discussed in Section 1. Their multiqueue network interface design [2] maps incoming flows to per-process queues with interrupt moderation, dropping packets for unregistered flows before an interrupt fires; at 5,000 packets/s it prevented roughly 93% of interrupts and reduced the critical task's additional runtime by about 80%. The interface was simulated rather than fabricated, and the paper does not establish whether ESP32-class radio hardware could support per-queue steering. Their journal treatment [3] adds a software route, performing early priority-based demultiplexing inside the driver and achieving a 7.5-fold reduction in per-packet processing for low-priority flows on a Zynq-7000.

2.2 Timing isolation on larger platforms

Outside this device class the finding is consistent: spatial separation combined with a resource regulator preserves real-time behaviour, and the platforms are large. Fischer et al. [4] measure control-task jitter on Linux-class industrial devices under wired floods and find core pinning effective, with a separate co-processor best; the attacked interface there is a physically separate NIC. Meszlényi et al. [5] hold DDS latency predictable under heavy interference using priority-based middleware scheduling, AF_XDP kernel bypass and dedicated NIC hardware queues on x86 with a PREEMPT-RT kernel. Hoornaert et al. [6] co-design CPU-time and memory-bandwidth isolation on a Cortex-A53 MPSoC, switching the interconnect from fair to fixed-priority arbitration on overload using a hypervisor and an FPGA-hosted memory scheduler. The mixed-criticality scheduling literature these sit within is surveyed by Burns and Davis [7], which treats criticality under benign timing assumptions rather than adversarial ones.

None of the mechanisms these works depend upon—a memory management unit, a hypervisor, performance-counter regulation, hardware queues—exists on a 240 MHz microcontroller. What remains available is core pinning and queue discipline, which is precisely why their sufficiency matters.

2.3 Radio-layer attacks on this device class

Deauthentication attacks against the ESP32 family are inexpensive and demonstrated. Kremser and Graffi [8] build deauthentication firmware for the ESP32-S3 itself, patching the compiled image because the vendor transmit path rejects such frames, and note reduced effectiveness against networks using protected management frames. Cayre et al. [9] reverse-engineer the wireless ROM components of the ESP32 family and inject, alter and jam link-layer traffic from software. Carbajal and Jodeiri Akbarfam [10] quantify deauthentication resilience across open, WPA2, PMF and WPA3 configurations. In all of this the dependent variable is whether the *link* survives; none measures whether the victim's *application* continues to meet its deadline. Mudraje et al. [11] reverse-engineer the closed-source ESP32-C3 Wi-Fi driver for static worst-case *energy* analysis, confirming that the driver is opaque and analysable only with substantial effort, but do not address timing or task interference.

2.4 Finding defects in embedded firmware

Fuzzing is the dominant automated technique for embedded firmware, and Yun et al. [16] survey the field. The hard problem is execution: firmware expects peripherals that a fuzzer does not have, so recent work concentrates on rehosting. Bley et al. [17] deduce the protocols a firmware image speaks and synthesise well-formed packets so that fuzzing reaches past the parser into deeper logic, and Tharindu

et al. [18] fuzz at the system level rather than per-peripheral. Our fuzzing is far more modest: a single command parser, exercised under sanitizers against host stubs, with no rehosting at all. We report it as a check on the parser rather than as a contribution to fuzzing.

The relevance here is negative, and it motivates Section 5. Three million sanitizer-backed iterations over that parser found nothing, and neither did the test suite, the adversarial campaign or exhaustive model checking—yet an unbounded drain sat in the safety path throughout. Techniques that execute the system can only find defects the system's own execution exposes. The two defects reported in this paper were found instead by examining the system's *assumptions*: one by response-time decomposition, one by ablation against a fair baseline.

2.5 Security and verification on the ESP32 application stack

Work on this platform measures the cost of MQTT over TLS on an ESP32-C3 [12] and runs machine-learning intrusion detection on-device [13]. Both optimise transport cost or detection quality rather than treating a safety deadline as the dependent variable, and an on-device detector is itself additional load on the contended chip. On-device deauthentication detection has been demonstrated previously, and we therefore claim no novelty for our detector; our contribution regarding it is the sensitivity analysis in Section 9. Krichen's survey of formal verification for the Internet of Things [14] covers model checking, abstract interpretation and deductive verification with Event-B, Maude and Simulink tooling, and records no use of TLA+ [15] for this device class.

Application-layer authentication for MQTT, placed above the broker precisely so that the broker need not be trusted, is an established and actively developed mechanism class, and we claim no novelty for it. Amirkhanova et al. [21] implement AES-GCM authenticated encryption on an ESP32 MQTT client — the same SoC and the same transport as this work — so that the broker relays only ciphertext it cannot read or alter undetected. Yu et al. [22] treat the broker as honest-but-curious and add timestamp freshness and renewable hash-chain anchoring. On the attack side, Alsabbagh et al. [19] demonstrate an adversary that connects to an MQTT broker as a legitimate client and issues unauthorized commands to an industrial system, and Zeghida et al. [20] survey weak and leaked credentials as a cause of unauthorized access. The principle that transport security does not supply application-layer authorization is therefore established, the attack class is published, and the mitigation class is published on this very platform. We make no claim of a new vulnerability. What we report in Section 9 is narrower and, we argue, still useful: a compile-time configuration guard that encodes that unsound inference, passes review without warning, and converts the known principle into the loss of a *safety* property in a system whose central claim is that the safety function stays live under attack.

3 System and Threat Model

3.1 System model

The device is a dual-core microcontroller with a single integrated radio. We write C_s for the core hosting the safety function and C_n for the core hosting all network activity. The safety function is a periodic loop that samples sensors, advances a finite state machine and drives outputs. Let t_d be the instant a physical intrusion asserts a sensor line and t_a the instant the siren is driven. The quantity of interest is the sensor-to-siren latency

$$L = t_a - t_d, \tag{1}$$

and the safety requirement is that $L \le L_{\max}$ for a deadline $L_{\max}$ fixed by the application.

Let Λ denote the offered load on the radio, in frames per second. Write F_Λ for the distribution of L under load Λ. The isolation property we wish to establish is

$$F_\Lambda \overset{d}{=} F_0 \quad \text{for all } \Lambda \text{ in the operating range}, \tag{2}$$

Table 1: Positioning against the closest work. Deadline degradation on this chip family has already been measured; what remains open is whether the only mitigation available on stock hardware is sufficient, under load that reaches the radio rather than the IP stack.

Work	Platform	Deadline measured	Radio load	Pinning as mitigation	Adversarial
Behnke et al. [1]	ESP32	yes	no	no	no
Behnke et al. [2]	Simulated NIC	yes	no	no	no
Behnke et al. [3]	Zynq-7000	yes	no	no	no
Fischer et al. [4]	Linux industrial	yes	no	no	yes
Meszlényi et al. [5]	x86 PREEMPT-RT	yes	no	no	no
Hoornaert et al. [6]	A53 + hypervisor	yes	no	no	no
Carbajal et al. [10]	Wi-Fi testbed	no	yes	no	yes
Kremser et al. [8]	ESP32-S3	no	yes	no	yes
Cayre et al. [9]	ESP32 family	no	yes	no	yes
This work	**ESP32**	**pending**[†]	**yes**	**yes**	**yes**

[†]Logical isolation is established here by model checking and simulation. The physical deadline measurement is specified in Section 10 but has not been performed, and no latency figure is claimed.

that is, the latency distribution under load is indistinguishable from the unloaded control. We decompose this into two claims that require different evidence:

Logical isolation. No control- or data-flow dependency exists by which network activity can block, delay or corrupt the safety path. This is a property of the program and is decidable by model checking and simulation.

Timing isolation. Contention for shared silicon—bus, cache, interrupt controller—does not shift F_Λ beyond the deadline margin. This is a property of the implementation on particular hardware and is decidable only by measurement.

Logical isolation is necessary but not sufficient for (2). This paper establishes the former and specifies the experiment for the latter.

3.2 Threat model

The adversary is within radio range, does not possess the device's shared key, and cannot physically access the device. The adversary may: transmit arbitrary 802.11 frames, including deauthentication and disassociation management frames; observe all traffic on the wireless medium, and therefore capture any message the device or its operator transmits; publish arbitrary messages to the device's command topic, either by reaching a shared broker or by operating one; and sustain any of these indefinitely.

This is adversary A, and it is the *primary* threat model: every claim this paper makes about the design, including the isolation property (2) and the formal results of Section 6, is a claim against adversary A unless explicitly stated otherwise.

Sections 9.3 and 9.3.1 additionally evaluate four *extended compromise* scenarios. These are not part of the primary model. They exist because a comparison of command-path defences is uninformative unless one asks what each defence costs when its own assumption fails, and because omitting them would leave the ablation resting on a transport baseline whose security we had never tested. Table 2 states them.

Scenarios B, C and D remain *remote*: broker credentials are bearer tokens, commonly shared across every client in a deployment and stored in plaintext in a configuration header, so an adversary can obtain one without ever touching the device. Such an adversary connects to the broker exactly as a legitimate client does and requires no cryptographic break.

Table 2: Adversary scenarios. A is the primary threat model; B through E are extended compromise analyses reported separately and never used to support a claim about the design.

	Additional capability	Access	Evaluated in
A	none (radio range only)	remote	throughout
B	one shared broker credential	remote	§9.3, §9.3.1
C	controller credential	remote	§9.3, §9.3.1
D	device credential	remote	§9.3, §9.3.1
E	device credential + command key	**physical**	§9.3, §9.3.1

Scenario E is different in kind and we keep it separate for that reason. Recovering the command key requires reading the device's flash, which requires physical access, so E abandons the remote-only assumption that A through D share. We do not mix the two: no result in this paper claims that the design resists an adversary with physical access, and E is reported only to show what the layered configuration does and does not buy once the key is gone. Where a conclusion depends on E, we say so.

The security of the signed-command path rests on assumptions about key management that we state rather than assume away. The command key must have been changed from the published default; it is symmetric, so every authorized publisher holds a key that can also forge, and it therefore exists in at least two places; there is no per-device derivation, rotation or revocation; and because this work does not enable the SoC's flash encryption or secure boot, the key is recoverable from the device by an adversary with physical access, JTAG or a flash dump. The claim we defend is correspondingly scoped: against an adversary holding broker credentials and arbitrary network position, but *not* the command key and *not* the ability to read device flash, application-layer authentication prevents unauthorized disarm, and the monotonic counter additionally prevents replay. We do not claim that a message authentication code is secure under broker compromise in general, and under key compromise it provides nothing.

Out of scope: physical tampering, side-channel key extraction, supply-chain compromise of the firmware image, and wideband jamming of the 2.4 GHz band, which defeats every wireless channel including the out-of-band fallback and against which the local siren is the only remaining defence. We note this limitation rather than claim coverage we do not have.

The assets to be protected, in priority order, are: the integrity of the alarm state, meaning that no unauthorised party can disarm the system; the timeliness of the local alarm response, meaning $L \leq L_{\max}$; and the delivery of notifications, which is best-effort by design.

4 Design

4.1 Domain separation

The design places the safety function on C_s and all network activity on C_n, and permits exactly three channels between them. Figure 1 shows the arrangement.

Three mechanisms enforce the separation, and each addresses a distinct failure mode.

Core pinning prevents the network task from occupying the safety core's execution slot. The network task is created with an explicit core affinity; the safety loop runs as the default application task on the other core.

Bounded queues prevent either side from exhausting the other's memory or blocking it. The event queue (safety to network) holds sixteen entries and the command queue (network to safety) holds four. On overflow the oldest event is discarded *and counted*, so loss is observable rather than silent; the published device state carries the drop count.

A zero-timeout status snapshot prevents priority inversion from reaching the safety side. The asymmetry here is the mechanism and is easy to overlook: the safety loop acquires the status mutex with a timeout of zero and simply skips the update if the network task holds it, while the network task may wait up to 10 ms. Contention can therefore delay the network side and never the safety side.

4.2 Command authentication

Remote commands carry an authentication tag and a counter. With shared key K and message $m = \texttt{VERB} \,\|\, \text{":"} \,\|\, c$ for counter c, the transmitted tag is

$$\tau = \mathrm{trunc}_{16}(\text{HMAC-SHA256}(K, m)) \,, \tag{3}$$

where HMAC follows RFC 2104 and truncation to 16 octets is permitted by its Section 5, giving a forgery probability of 2^{-128} per attempt. A command is accepted iff

$$\mathrm{ct_eq}\big(\tau, \tau'\big) \;\wedge\; c > c_{\text{stored}}, \tag{4}$$

where τ' is the recomputed tag, $\mathrm{ct_eq}$ is a constant-time comparison that accumulates differences rather than returning early, and c_{stored} is the counter held in non-volatile storage. Both conjuncts are necessary and they defend different things: the first defeats forgery, the second defeats replay. Section 9 shows empirically that a design with only the first is broken by capture-and-replay.

Two placement decisions matter more than the construction, which is standard.

The expensive verification runs on C_n, before the command queue. A flood of forged commands is therefore rejected entirely within the network domain and consumes no safety-core time; only a verdict and an enumerated verb cross the boundary. Figure 2 shows the path.

The counter is committed to non-volatile storage *before* the command is acted upon. Were the order reversed, an interruption between action and commit would leave the command replayable. In the chosen order the worst case is that a counter value is consumed without the command executing, which fails safe: a legitimate sender increments and retries, while an adversary gains nothing.

4.3 Supporting mechanisms

A passive detector counts 802.11 deauthentication and disassociation frames in a sliding window and raises an event when the count reaches a threshold, subject to a cooldown that bounds the report rate. Writing $N(t)$ for the number of such frames in $(t - W,\, t]$, the detector reports at t iff

$$N(t) \geq T \;\wedge\; t - t_{\text{last}} > C. \tag{5}$$

The detector never transmits. The cooldown is not cosmetic: without it the detector could itself flood the bounded event queue, converting a radio attack into a denial of service against the notification path. Parameters T and W were chosen by the sweep in Section 9, which overturned our initial choice.

When the broker is unreachable the device can send a signed, counter-protected alert to a second device over ESP-NOW, which operates without an access point. Optional sensor fusion requires agreement between a passive infrared sensor and a millimetre-wave radar within a window W_f, reducing single-sensor false alarms. Armed state and the command counter persist in non-volatile storage, so a power interruption while armed does not silently disarm the system.

5 Response-Time Analysis

Measurement tells us whether interference occurs; it does not tell us which mechanism produced it, nor what a different design would do. A static decomposition answers the complementary question: of the interference a safety task on this architecture can suffer, how much does this design remove by construction, and how much is irreducibly a property of the silicon?

5.1 Decomposition

A confirmed detection requires the sensor line asserted for T_{conf}, after which the next loop pass advances the state machine and drives the output. Writing R for the worst-case response time of one loop pass,

$$L \;\leq\; T_{\text{conf}} + 2R, \tag{6}$$

since at most one pass observes the confirmed edge and at most one further pass drives the output. The quantity to bound is therefore R, which decomposes as

$$R \;=\; C_{\text{own}} \;+\; B \;+\; I_{\text{isr}} \;+\; I_{\text{mem}}, \tag{7}$$

where C_{own} is the loop's uncontended computation, B is blocking on resources shared with the network task, I_{isr} is preemption by interrupt handlers executing on the safety core, and I_{mem} is stall cycles from contention for the shared bus, cache and flash.

5.2 Three terms settled by construction

$B = 0$. The only resource the safety loop shares with the network task is the status snapshot. The safety loop acquires it with a timeout of zero, so the call cannot block: it either succeeds immediately or returns, deferring the update to the next pass. The network task, by contrast, waits up to 10 ms. Therefore $B = 0$ unconditionally, independently of what the network task does, including holding the mutex indefinitely. Priority inversion can delay the network side and cannot delay the safety side. This is precisely the property SAFETYCANALWAYSSTEP verified in Section 6. The cost is that a status snapshot may be skipped, which is acceptable because the snapshot feeds notification, which is best-effort, while the siren is driven from state the safety loop owns outright.

C_{own} is constant-time. Every stage of the loop performs $O(1)$ work. Neither queue can block, both being accessed with a zero timeout; the command drain is bounded by the queue capacity; the display stage, which dominates C_{own} because it performs an I^2C transaction, is rate-limited to once per 250 ms. The deauthentication detector could in principle emit one event per observed frame; it does not, because the cooldown in (5) admits at most one report per 10 s. Without that cooldown a radio flood would become an event flood and the bound would fail, so the cooldown is a timing mechanism as much as a usability one.

The residual. I_{isr} and I_{mem} cannot be bounded from source. The Wi-Fi driver is closed-source; Mudraje et al. [11] required substantial reverse engineering to model its resource consumption for worst-case *energy* analysis and did not address timing. Interrupt core affinity is set by the vendor SDK and not guaranteed, and the memory subsystem's contention behaviour is not published. Hence

$$R \;=\; \underbrace{C_{\text{own}}}_{O(1)} \;+\; \underbrace{0}_{B} \;+\; \underbrace{I_{\text{isr}} + I_{\text{mem}}}_{\text{hardware, unmeasured}} \quad . \tag{8}$$

Three of four terms are settled. The residual is a single composite hardware quantity, and the experiment of Section 10 measures exactly it: the difference between the unloaded and loaded latency distributions isolates $I_{\text{isr}} + I_{\text{mem}}$, because every other term is held constant by construction. Without this decomposition a measured shift could be attributed to any of four causes; with it, only one remains. This is what makes the pending measurement interpretable rather than merely suggestive.

5.3 A falsifiable prediction

We state a prediction before measurement so that it cannot be adjusted afterwards. Given $B = 0$, constant-time C_{own}, and the single-digit-percent cycle loss Behnke et al. [1] measured for IP-layer load, we predict a median latency shift under 10 ms against a T_{conf} of 150 ms, that is under 7% of the confirmation window, with no missed detections. The prediction is refuted by a median shift above 10 ms, by a heavy upper tail indicating occasional interrupt storms even where the median holds, or by any missed detection.

5.4 A defect the analysis exposed

Establishing that every stage is constant-time required checking each one. One was not. The command queue was drained with an unbounded loop, so the safety loop had no static bound on its own execution

Table 3: Properties verified by TLC. Both specifications are mutation-tested; see Section 6.3 for a property that was vacuous in the shipped specification.

Property	Type	Statement
NoUnauthRemoteDisarm	invariant	No unauthenticated remote command causes a transition to the disarmed state
CounterMonotonic	property	The replay counter never decreases
NoAlarmWhenDisarmed	invariant	The siren is never active in the disarmed state
SafetyCanAlwaysStep	invariant	The safety step is enabled whenever work remains
NoNetworkWrite	invariant	The network task never writes alarm state
QueueBounded	invariant	The event queue never exceeds its capacity
NoSilentLoss	invariant	produced = delivered + queued + dropped
EntryLeadsToResolve	liveness	An entry delay always ends in alarm or disarm
SafetyProgress	liveness	The safety function makes progress

time: the network core can enqueue while the safety core dequeues, and termination rested on the drain rate exceeding the fill rate. That argument is probabilistic, and weak in the default build, where enqueueing costs only a parse with no tag verification to throttle it. The asymmetry was also inverted, since the network task already bounded its own event drain while the safety task did not. We bounded the drain to the queue capacity, so a full queue still clears in one pass and later arrivals wait at most one loop period.

We record this because of what it says about method. The defect was invisible to every other technique applied here: the test suite passed, the fuzzer found nothing, the adversarial campaign reported no violation, and the TLA+ models have no notion of execution time and so could not have caught it. It was found by asking what bounds each stage, and finding that one stage had no answer.

6 Formal Verification

We specify two models in TLA+ [15] and check them exhaustively with TLC.

6.1 State machine and command path

The first specification models the five alarm states, the transitions between them and the command rules. Over all 25 reachable states at the configured counter bound, TLC verifies that no unauthenticated remote command disarms the system, that the replay counter never decreases, that the siren is never active while disarmed, and that the entry delay and the alarm always resolve rather than deadlocking.

6.2 The isolation argument

The second specification is the one that carries the paper's design claim. It models both tasks, with the network task made maximally adversarial: it may become busy, stall for an unbounded time, fail to publish, crash and restart, and it is granted *no fairness*. Weak fairness is asserted for the safety step alone:

$$Spec \triangleq Init \wedge \Box[Next]_{vars} \wedge \mathrm{WF}_{vars}(SafetyStep). \tag{9}$$

Table 3 lists the verified properties. TLC explored 2,686 states, 660 of them distinct, to a graph depth of 26, with no violation.

6.3 Mutation testing, and a vacuous invariant we shipped

A model checker reports no violation for a vacuous specification as readily as for a sound one, so each property must be shown to have force. Allowing the network task to write alarm state violates NoNetworkWrite, as expected.

NoUnauthRemoteDisarm did not have force, and we report this because the reason is the subject of this paper. The specification modelled an unauthenticated remote command with an action named `RemoteUnauthDisarmRefused` whose entire body was `UNCHANGED`, carrying a comment asserting that this is "exactly as the firmware does". The flag `badDisarm` was initialised false and left unchanged by every action in the specification, so it could never become true. The invariant NoUnauthRemoteDisarm, which states simply that `badDisarm` is false, was therefore unfalsifiable by construction. The mutation test for it had been performed by hand-editing a local copy; the specification we shipped and checked in continuous integration was vacuous for that property.

The cause is the same unsound inference the compile guard encoded. Both artefacts *assumed* that an unauthenticated remote disarm is refused, rather than deriving it, so neither could detect that a build configuration existed in which it is not. A model that embeds the assumption under test cannot test it, and this is why exhaustive model checking did not find a defect the ablation did.

The configuration is now a parameter of the specification. With `AllowUnauthDisarm` = FALSE the command is refused and the invariant holds non-vacuously, the violating action being reachable under the other valuation; with `AllowUnauthDisarm` = TRUE the command takes effect exactly as the `ALLOW_REMOTE_DISARM` build does, and TLC returns a three-state counterexample in which the system is armed by button and then disarmed by an unauthenticated remote command. Both configurations run in continuous integration, and the defective one is required to *fail*: a run in which it passes means the invariant has become vacuous again. The defect is therefore now reproduced formally as well as experimentally.

6.4 What the models do not cover

The abstraction boundary deserves explicit statement, because it is exactly where the research question lives. The network task is modelled as a set of discrete actions, not as a source of *timing interference*. TLC establishes that within this abstraction no control- or data-flow dependency can block the safety step. It says nothing about contention for shared silicon, which is a property of the implementation rather than the model. Model checking here bounds the logical design space; it does not substitute for measurement.

7 Implementation

The firmware comprises approximately 1,100 lines of C++ for the main application, together with four self-contained headers. These provide a portable SHA-256 and HMAC-SHA256 implementation, checked against the published RFC 4231 test vectors; the deauthentication detector; the signed out-of-band alert message; and the sensor-fusion logic. The three latter headers contain pure logic with no platform dependency, which allows them to be tested on a host without hardware.

7.1 Verification infrastructure

The unmodified firmware compiles against host models of Arduino, FreeRTOS, Wi-Fi, MQTT, non-volatile storage and the display. The firmware source is not altered for testing, which is what makes the simulation results meaningful. The suite runs nineteen scenarios at three clock start values across two vendor framework versions, plus build-configuration and header-synchrony checks, for 120 checks in total.

The three clock start values include one immediately below the 32-bit wrap of the millisecond counter. All elapsed-time comparisons are written in the wrap-safe form $(\texttt{uint32_t})(now - since) \geq interval$ rather than $now \geq since + interval$, which fails at the boundary. This is not a theoretical concern: a defect of exactly this class, under which the siren could remain active after approximately 25 days of uptime, was found by this method during development.

Table 4: Build cost, as reported by the compiler in continuous integration. Static sizes only; no run-time or energy cost has been measured.

Framework version	Flash (B)	of budget	RAM (B)	of budget
arduino-esp32 2.0.17	793,433	60.5%	46,372	14.2%
arduino-esp32 3.3.12	968,221	73.9%	49,876	15.2%

7.2 Resource cost

Table 4 gives flash and RAM consumption as reported by the compiler for both framework versions. The security features added in the hardened configuration cost approximately 3 kB of flash and 170 bytes of RAM. We emphasise that this is a static measure: no CPU-time or energy cost has been measured, and both remain open.

8 Experimental Methodology

8.1 Evidence categories

Results in this paper fall into three categories and we label each one, because they are not interchangeable. *Formal verification results* come from exhaustive model checking. *Simulation results* come from the unmodified firmware driven by a seeded harness on a host. *Analytical results* follow from the construction, such as the 2^{-128} forgery probability. No result in this paper is a physical hardware measurement; the single exception is the build cost of Table 4, which a real compiler measured but which describes the binary rather than the running device.

8.2 Hypotheses

H1 (logical isolation). No execution of the system exists in which network activity blocks, delays or corrupts the safety path, and no forged, stale or malformed command is accepted. *Testable by model checking and simulation.*

H2 (timing isolation). $F_\Lambda \overset{d}{=} F_0$ for radio-layer load Λ across the operating range. *Requires hardware; not tested here.*

Given [1], we do not expect H2 to hold exactly. The informative question is the magnitude of any shift relative to the deadline margin, and whether queue discipline bounds it.

8.3 Determinism and reproduction

Nothing in the evaluation is collected as a dataset. Inputs are generated deterministically from fixed seeds, which is stronger for this purpose than distributing a file: an independent researcher regenerates the identical inputs by running the scripts. Each trial of the adversarial campaign and the ablation executes in a forked child process, because the firmware's globals are file-scope and would otherwise leak between trials. This was not a stylistic choice: before it was introduced, state leakage produced a spurious invariation violation at zero attack rate.

The campaign and fuzzing pipeline has been verified to regenerate its summary files byte-identically from the committed seeds.

8.4 Metrics

Table 5 defines every metric reported in Section 9. Two of them changed during this work after the harness defects described in Section 12, so we state their definitions precisely rather than relying on their names.

Table 5: Evaluation metrics and their definitions.

Metric	Definition
Safety latency	Simulated milliseconds from confirmed sensor assertion to entry-delay entry
Command availability	Trials in which a legitimate command issued during the flood took effect
Integrity failure	Trials in which an adversary's message caused a state change
Replay defeat	Trials in which the adversary obtained an accepted command using only captured or forged material
Invariant violation	Trials violating any safety invariant
Accepted fuzz input	Inputs changing state or advancing the counter without a valid tag
False-positive rate	Benign runs in which the detector reported at least once
Detection latency	Simulated milliseconds from attack onset to first detector report

Table 6: Trial accounting. Every figure is derived from the named committed artefact; no count in this paper is stated independently of these files.

Experiment	Committed artefact	Count
E1/E2 adversarial campaign 500 per rate $\times$ 6 rates $\times$ 2 builds	`summary.json`	6,000 trials
E3 ablation 300 per configuration $\times$ 11	`ablation.jsonl`	3,300 trials
Total adversarial trials		**9,300**
E4 command-parser fuzzing	`fuzz.json`	3,000,000 iterations
E5 detector sensitivity	`sensitivity.jsonl`	180 grid cells
E6 broker access control	`broker_acl.jsonl`	8 scenarios

8.5 Experimental configurations

The adversarial campaign runs 3,000 trials per build (500 at each rate) across six attack rates from 0 to 20,000 messages/s. The adversary injects forged tags signed with a key it does not hold, replays of messages captured on the wire, bare verbs carrying no credential, and random bytes. The ablation runs 300 trials per design over the same rates with identical seeds. The fuzzer runs 3,000,000 iterations over six input classes under AddressSanitizer and UndefinedBehaviorSanitizer. The sensitivity sweep runs 2,000 benign trials of ten simulated minutes per grid cell.

Table 6 reconciles every count reported in this paper against the committed raw data, so that a reader can check the totals rather than take them on trust. Each row names the file the count is derived from.

The eleven configurations of E3 comprise six command-path designs (Table 7) and five further credential-compromise configurations (Table 8); the transport arms appear in both tables, which is why the two tables together name more than eleven rows while the dataset holds exactly 300×11 trials.

9 Results

9.1 Adversarial campaign

Across 6,000 trials no adversary disarmed the device and no safety invariant was violated. Without command authentication the adversary changed the system state in all 2,500 flooded trials; with authentication, in none. The availability result is the more informative one: with authentication a legitimate command took effect in all 3,000 trials regardless of attack rate, while in the baseline flooding starved it in every flooded trial.

We report no hypothesis test on these data. Because the harness advances a logical clock there is no

Table 7: Command-path ablation. 300 trials per design, identical adversary and seeds. "Operator admitted" counts operator publish attempts that reached the state machine while the adversary flooded, over five attempts per trial; "defeated by replay" counts attacked trials in which the adversary obtained an accepted command using only captured or forged material.

Design	Operator admitted	Defeated by replay	Attacker disarmed	Alarm fired
No command defence	1250/1250	250/250	0	300/300
Rate limit, 5 commands/s	737/1250	250/250	0	300/300
Static shared token	1250/1250	250/250	0	300/300
TLS + broker credentials	1250/1250	0/250	0	300/300
TLS, credentials leaked	1250/1250	250/250	246	50/300
HMAC-SHA256 + counter	**1250/1250**	**0/250**	**0**	**300/300**

contention, the safety latency is a deterministic 151 ms in every trial—the sensor confirmation window plus loop period—and its variance is identically zero. Any test on two constant-valued samples is trivially satisfied and carries no information. We pre-register the analysis for the hardware measurements instead: a two-sided Mann–Whitney U test with tie correction, chosen because latency distributions on a real-time operating system are not normal, with the implementation committed in advance.

9.2 Command-path ablation

To avoid resting on a baseline that is defeated by construction, we ran the same adversary against six command-path designs on identical seeds: no command defence; a token-bucket rate limiter admitting five commands per second; a static shared token carried in the payload; TLS with broker credentials that the adversary does not hold; the same TLS configuration with the credentials leaked to the adversary; and the signed-counter path. The rate limiter and the token are modelled as edge filters, which is where a developer would place them in practice. The two transport arms are compiled from identical source with identical flags and differ only in whether the adversary possesses a working broker credential, which makes the comparison a controlled one. Table 7 reports the outcome and Fig. 3 plots the three properties it separates.

The result separates three properties that are easily conflated. A rate limiter cannot distinguish a legitimate command from a flood, so it discards both: it admits 59.0 % of operator attempts against 100 % for every other design (Fisher exact, one-sided, $p = 3.5 \times 10^{-184}$), while leaving integrity entirely untouched. Its cost is not monotonic in attack intensity, for a reason we measured rather than assumed: an arrival that finds the bucket below the admission threshold is rejected *without consuming a token*, so an extreme flood pins the bucket just under that threshold while a moderate flood drives it lower. A static shared token leaves availability intact, because the edge filters the flood cheaply, but it travels in plaintext, so a single captured command defeats it. Only the monotonic counter prevents replay; a message authentication code alone would behave as the token row does.

The two transport arms isolate the one configuration in which the safety function fails. With broker credentials held, TLS is a strong baseline: the adversary is refused at the broker, never obtains an accepted command, and the alarm fires in all 300 trials. With the same build and the credentials leaked, the adversary disarms the system in 246 of 300 trials and the alarm fires in only 50 — 16.7 %, 95 % Wilson CI [12.9 %, 21.3 %], against 100 % for the control (Fisher exact, one-sided, $p = 9.8 \times 10^{-119}$). The 50 successes are exactly the trials in which the adversary was silent. The cause is an unsound compile-time guard, analysed in Section 11: the firmware accepted "TLS enabled and a broker login configured" as sufficient to permit unauthenticated remote disarm. It is not. The broker terminates TLS and re-originates every message, so an arriving command carries no evidence of its publisher, and the macro recording that a broker login is configured describes only this device, not any publisher's authorization. We make no claim that TLS is broken; Section 11 states precisely what it does and does not protect.

Table 8: What each control protects, by which secret the adversary holds. 300 trials per row, identical adversary and seeds. "Locked out" reports operator commands admitted while the adversary is active. Broker-side access control and application-layer authentication fail under *different* compromises, so neither dominates.

Controls in place	Secret the adversary holds	Disarm	Alarm fired	Operator admitted
TLS only	none	no	300/300	1250/1250
TLS only	any broker credential	yes	50/300	1250/1250
TLS + per-client ACL	the one shared credential	yes	50/300	1250/1250
TLS + per-client ACL	controller credential	yes	50/300	1250/1250
TLS + per-client ACL	device credential	no	300/300	1250/1250
HMAC + counter	any broker credential	no	300/300	1250/1250
HMAC + counter	controller credential and command key	yes	50/300	**0/1250**
TLS + ACL + HMAC	device credential and key (flash dump)	**no**	**300/300**	**1250/1250**

9.3 Broker access control against application-layer authentication

A reviewer may fairly object that the transport arm above models a broker with no topic-level access control, which is weaker than a careful operator would deploy. We therefore added five further arms, all on the same seeds, that vary the broker's access-control list and which secret the adversary holds. The ACL is modelled as a Mosquitto-style binding of each username to the topic patterns it may publish and subscribe to. We stress that none of these is a deliberately weakened configuration: for the alarm to function, *some* identity must hold write permission on the command topic and the device must hold read permission on it, so which identity the adversary steals is what decides whether the ACL contains the compromise. Fig. 4 shows the permissions modelled, and Table 8 with Fig. 5 report the outcome.

Four readings follow, and the last three cut against this work's own mechanism.

First, a per-client ACL is *not* a weaker substitute for application-layer authentication. It contains a compromise of the *device's* credential completely, which matters for an embedded product precisely because the device is the physically accessible component and its broker credentials sit in a readable flash image. It does nothing against a compromised controller, because that is the identity the ACL must authorise to publish commands.

Second, a shared credential defeats the ACL entirely. The ACL must grant the single identity write access to the command topic for the system to work at all, so the configuration collapses to the no-ACL case. Credential separation, not ACL syntax, carries the protection, and shared credentials are common practice in deployed IoT fleets.

Third, a MAC does not survive key compromise, and on this platform the key is in the same flash image as the broker credentials. With the command key dumped, the signed path is defeated exactly as the unsigned one is. Only the combination holds: an adversary who dumps the device obtains a key it can sign with *and* a credential the broker will not let it publish commands under. The configuration that survives every row of Table 8 is layered, not cryptographic.

9.3.1 Validation against a real broker

The arms above model broker admission as a per-identity permission. To check that the model is faithful rather than convenient, we repeated the scenarios against an actual broker: Mosquitto 2.0.18 over TLS 1.2 with a private certificate authority, password authentication, and per-client ACL files. This is the one experiment in this paper that is a *testbed measurement* rather than a simulation.

One methodological point governs how the results are read. MQTT 3.1.1 has no return code meaning "refused by policy": the broker acknowledges a QoS 1 publish and then silently discards it. A `PUBACK` is therefore not evidence of delivery, and we observed exactly that—every denied publish in Table 9 was acknowledged. Each scenario is decided instead by whether a subscriber holding the device's identity actually received the message, which is also how the real firmware would learn of it, namely not at all.

The broker reproduced every prediction of the model. It refused the connection outright in scenario

Table 9: Access-control scenarios against Mosquitto 2.0.18 with TLS and per-client ACLs. Testbed measurement. Scenario letters match the threat model in Section 3.

Scenario	Adversary uses	Connected	Delivered	Executes
control	controller (legitimate)	yes	yes	yes
A	no credentials	**no**	no	no
C	controller credential	yes	yes	**yes**
D	device credential	yes	**no**	no
E	device credential + key	yes	**no**	no
E′	controller credential + key	yes	yes	**yes**
B	shared credential (shared ACL)	yes	yes	**yes**

A; it delivered the adversary's command when the stolen identity was the controller's (C) or the single shared account (B); and it logged `Denied PUBLISH` and dropped the message when the stolen identity was the device's (D and E). Scenario E is the one that matters for the layered argument, and E′ is its control: in E the adversary holds a flash dump, so its command carries a signature that *does* verify under the real command key, and the broker stops it anyway; in E′ the identical signature published from an identity the ACL permits is delivered and would execute. The signature is therefore valid in both, and the ACL is what distinguishes them. No arm of the simulation was contradicted.

Two caveats remain. The broker ran on loopback with a private CA rather than in a deployed network, and the scenarios exercise ACL semantics rather than broker performance: no timing, throughput or handshake cost is measured or reported from this testbed. What it establishes is that the access-control behaviour the simulation assumed is the behaviour a real broker exhibits, which is what the simulation needed and did not previously have.

9.3.2 The replay counter as a lockout

Fourth, and unanticipated, under key compromise the monotonic replay counter becomes a lockout against the owner. Counters are not secret—they travel in plaintext in every signed command—so an adversary holding the key can sign with a counter far above the operator's. The device persists it, and every subsequent operator command is then rejected as a replay. In that arm the operator is admitted in **0 of 1250** attempts across all attacked trials: the adversary both disarms the alarm and denies the legitimate user any control. Recovery requires a key change and a counter reset, neither of which this firmware supports remotely. The mechanism that defeats replay therefore creates a denial-of-service channel the moment its key assumption fails, which we report because it bounds the claim made in Section 3 rather than supporting it.

Apart from the compromise arms, the alarm fired in every trial of every design. This is the isolation property behaving as intended: the safety function does not depend on the command path, so no command-path design choice affects whether the siren sounds. The leaked-credential arm is the single exception, and it is an authorization defect rather than a timing or resource failure, which locates the one place where the isolation argument does not hold.

9.4 Detector parameter sensitivity

The detector's threshold and window had a stated rationale but no measurement. Sweeping them against a Poisson model of benign management-frame arrivals showed that our initial threshold of eight was wrong. Access points emit deauthentication and disassociation frames in normal operation during roaming, idle timeout, load balancing and reboot; at 60 frames per minute, representing a busy multi-access-point environment, a threshold of eight fired spuriously in 79.9% of ten-minute runs. Table 10 gives the trade-off surface.

We revised the firmware default to twelve, which reduces the false-positive rate to 0.25% while still detecting every attack rate swept with no misses, at a cost of approximately 214 ms of additional

Table 10: Detector sensitivity. False-positive rate is measured at 60 benign frames per minute; detection latency is the median at the stated attack rate. The deployed configuration is shown in bold.

Threshold	Window (ms)	FP rate	Detect, 20 f/s	Detect, 5 f/s	Missed, 5 f/s
5	2000	1.0000	234 ms	948 ms	0/500
8	1000	0.0355	405 ms	2326 ms	0/500
8	2000	0.7985	383 ms	1520 ms	0/500
12	**2000**	**0.0025**	**597 ms**	**2673 ms**	**0/500**
12	5000	0.9450	577 ms	2348 ms	0/500
20	2000	0.0000	1004 ms	26137 ms	279/500
20	5000	0.0005	958 ms	3852 ms	0/500

detection latency at 20 frames per second. A detector that reports frequently on a healthy network trains its owner to ignore it, and on this device each report consumes an event-queue slot; the additional latency is a reasonable price.

We are explicit that the benign arrival model is an *assumption*, not a measurement. No capture of real management-frame traffic was available to us. These results therefore characterise the detector's logic against a stated model and are not a false-positive rate for any real network. Obtaining one requires a passive capture on a production network and is future work.

9.5 Fuzzing

The command parser is the one input a network peer fully controls. Across 3,000,000 iterations under AddressSanitizer and UndefinedBehaviorSanitizer the fuzzer found no memory error, no undefined behaviour, and no input that changed state or advanced the counter. We read this as evidence of memory safety and parser robustness rather than of authentication soundness: against a 128-bit tag, zero forgeries is the expected outcome and is not independent evidence.

One episode is worth recording as a methodological caution. An early run reported an accepted input. Investigation showed the mutation had flipped the case of a hexadecimal digit, which decodes to identical tag bytes, so the input was the genuine command and acceptance was correct behaviour. The corpus now signs its mutation seed with a deliberately incorrect key so that no mutation can produce a valid command by accident. Had this not been chased down it would have become a false claim.

10 Independent Hardware Validation Framework

The results above establish H1. They cannot establish H2, because the simulated clock does not model contention for the bus, the cache or the interrupt controller, and that contention is precisely the phenomenon in question.

We did not have access to the required hardware. Rather than leave H2 as an unqualified gap, we treat the validation apparatus itself as a deliverable: firmware for the measuring instrument, wiring, experimental conditions, collection and analysis scripts, pre-registered statistics and explicit falsification criteria, all public, so that any laboratory with two inexpensive boards can perform the measurement and report it. The decomposition of Section 5 is what makes the result interpretable when they do, since it establishes that a measured shift can be attributed to $I_{\mathrm{isr}} + I_{\mathrm{mem}}$ and to nothing else.

We state plainly that this is a framework and not a result. No physical measurement appears in this paper.

10.1 Apparatus

Figure 6 shows the arrangement. The device under test runs the unmodified firmware. A second, independent microcontroller acts as the measuring instrument: it drives the sensor line, timestamps that

edge with its own hardware timer, and timestamps the rising edge of the siren line. The measuring clock is therefore never the clock under load, which is essential: a device cannot credibly measure its own latency while that latency is the quantity being perturbed.

10.2 Conditions

Four conditions, each compared against the unloaded control:

A. Idle. No network activity. Establishes F_0 and the instrument's own noise floor.

B. Legitimate load. Normal MQTT traffic with the broker reachable.

C. Command-interface stress. High-rate publication on the command topic.

D. Radio-layer stress. Deauthentication and disassociation frames at the MAC layer.

Conditions A through C require only the two boards and a probe. Condition D requires an authorised isolated or shielded radio environment and institutional permission, and is the condition that distinguishes this work from prior IP-layer measurements.

10.3 Analysis plan, pre-registered

Each loaded condition is compared against condition A using a two-sided Mann–Whitney U test with tie correction. For samples of size n_1 and n_2 with rank sum R_1,

$$U_1 = R_1 - \frac{n_1(n_1+1)}{2}, \qquad U = \min(U_1,\, n_1 n_2 - U_1), \tag{10}$$

$$\sigma_U^2 = \frac{n_1 n_2}{12}\left[(n+1) - \frac{\sum_i (t_i^3 - t_i)}{n(n-1)}\right], \qquad n = n_1 + n_2, \tag{11}$$

with $z = (U - n_1 n_2/2)/\sigma_U$ and a two-sided p-value from the normal approximation. The test is non-parametric because latency distributions on a real-time operating system are not normal. The bracketed term in (11) is the tie correction, which matters here because quantised timer readings produce many ties. The implementation is committed in advance of any measurement, and we will report effect size and the full distribution rather than a p-value alone.

10.4 Ethics

The radio-layer condition is to be performed only on equipment owned by the experimenter, in an isolated or shielded environment, with institutional permission obtained beforehand. No third-party network or device is targeted. Neither this paper nor its public repository contains any attack-generation procedure or tooling. We note that deauthentication tooling for this chip family is already published [8, 9]; we cite that work as evidence the threat is realistic while declining to add to it.

11 Discussion

11.1 What the evidence supports

The formal and simulation results together establish that the design contains no path by which network activity reaches the safety function logically. The TLA+ isolation model is the stronger of the two, because granting the network task no fairness means the property holds under every possible interleaving rather than the ones a harness happens to generate. The campaign and fuzzing results confirm that the implementation matches the specification.

We state plainly what the leaked-credential arm does *not* establish. That transport security does not supply application-layer authorization is a textbook principle; the rogue-authenticated-client attack

is published [19]; and application-layer authentication for MQTT is an established mitigation class, implemented on this very SoC with this very threat model [21, 22]. We claim no new vulnerability, no new attack, and no novelty for authenticated MQTT commands on an ESP32. What the arm contributes is a controlled, seed-matched measurement that an unsound *compile-time guard* converts that known principle into the loss of a safety property, and a baseline fair enough that the comparison against the signed-counter path is worth making at all: without it, a reader could reasonably object that a realistic deployment would use TLS with broker credentials and need no application-layer authentication. The transport control shows that objection is correct exactly as long as credentials hold, and the leaked arm shows where it stops being correct.

The ablation is the result we consider most likely to be useful to other designers, because it is the least predictable. It is tempting to evaluate a command-path defence by asking whether legitimate commands survive an attack. Both the static token and the signed-counter design pass that test identically, and a designer stopping there would conclude they are equivalent. They are not: one is defeated by a passive eavesdropper, and the measurement that distinguishes them is capture-and-replay rather than throughput.

11.2 Why the design behaves as observed

The availability result follows from where verification happens rather than from cryptography. Because the tag is checked on the network core before the bounded command queue, a flood of invalid commands is discarded without ever occupying a queue slot. The queue therefore retains capacity for the one valid command, and admission is decided by validity rather than arrival order. A rate limiter inverts this: it decides by arrival order, which an attacker controls.

11.3 A compile-time guard that encodes an unsound inference

The leaked-credential arm fails for a reason worth stating precisely, because the defect is not in a cryptographic primitive but in a build condition. The firmware permitted unauthenticated remote disarm whenever TLS was enabled *and* a broker login was configured. Read as an implication, that guard asserts: TLS enabled and a broker username supplied, therefore every message delivered on the command topic originates from an authorized principal. Both conjuncts fail to support the conclusion, and for different reasons.

TLS, as configured, validates the *broker's* certificate. The device presents no client certificate, so its own identity rests entirely on username and password; and because MQTT is a store-and-forward protocol in which the broker terminates TLS and re-originates each message, every publisher holds a separate session. A command arriving over the device's session carries no cryptographic evidence about who published it. TLS secures a hop; it does not convey end-to-end provenance across a relay. Fig. 8 contrasts the two builds. The second conjunct is worse: the macro recording that a broker login is configured is set by the presence of a username in the device's own build, so it is a fact about the *subscriber*, not about any publisher's authorization. The guard used the subscriber's credential state as a proxy for every publisher's authority, which is a category error. Even with a correct broker access-control list, authorization is then enforced outside the device's trust boundary and is only as strong as credential secrecy.

We emphasise that nothing here bears on the security of TLS, and we do not claim it is broken. TLS protects the confidentiality and integrity of the device-to-broker hop, authenticates the broker, and prevents on-path injection into that hop. It does not authenticate a publisher end to end, it has no notion of what a command means, and it offers nothing once a valid credential is in an adversary's possession — at which point the adversary simply *is* a legitimate client. Transport security and application-layer authorization are orthogonal, and no quantity of the former supplies the latter.

Fig. 7 sets the two guards side by side. The minimal correct repair is to delete the unsound branch rather than to add a mechanism. Unauthenticated remote disarm now requires an explicit acknowledgement macro named for what the builder is accepting, which the research baseline defines so the original behaviour remains reproducible. We separately refuse to compile a signed-command build that still

uses the repository's published demo key, since authentication that verifies against a published key is not authentication, and such a build would log every command as authenticated while any reader of the repository could forge one. Three negative build checks assert that both guards continue to reject the unsafe configurations. We deliberately did not add timestamp freshness, nonce challenge-response, or key rotation: the first requires a trusted clock the device does not have, the second costs a round trip on an at-most-once path, and the third is a fleet-management concern. Adding them would raise the contribution count without evidence to support it.

11.4 Relationship to prior measurement

Our work complements rather than contradicts [1]. They established that on-chip network activity reaches a critical task on this hardware even with cross-core separation; we accept that result and address the question it leaves open, namely whether the mitigations available on stock hardware are adequate, under the load class that actually threatens a wireless alarm. Their proposed mitigations require either hardware that does not exist [2] or a platform an order of magnitude larger [3]. If our pending measurement shows that pinning with queue discipline holds the deadline under radio-layer load, that is a useful positive result for a very large installed base. If it shows the opposite, that is a more important result still, because it would indicate that the standard advice given to developers of such devices is insufficient and that the installed base is exposed.

12 Threats to Validity

Construct validity. This is the most serious threat. The dependent variable in simulation is a logical latency measured on a clock the harness itself advances. We call it safety latency, but it is not the physical quantity of interest, and the divergence between the construct and the measure is the reason Section 10 exists.

Internal validity. Largely controlled. Seeds are fixed, the pipeline regenerates byte-identically, and continuous integration re-runs a reduced version on every commit. Per-trial process isolation was introduced specifically because global state leakage had produced a spurious violation. Four further harness defects were found during the ablation and are documented in full: the adversary could act during the experimenter's own control actions; it could leave the system mid-transition so that the intrusion phase was skipped; the operator availability metric could not distinguish the operator's own arm command from the adversary's in any arm that admits adversary commands; and a single-command measurement of availability measured token-bucket phase rather than availability. Two of the four flattered the conclusion the experiment was built to support, two were found only during the final verification pass, and the campaign was re-run from scratch after each correction. None affected the safety or disarm results, which are read from directly observed events carrying explicit source tags rather than inferred. We record them because three of the four produced entirely plausible numbers, and the first was detected only because results were broken out by attack rate rather than aggregated. The residual threat is that the host models and the firmware share an author, so a common misconception would be invisible to both.

External validity. Limited, and we scope our claims accordingly. All results concern one chip, two framework versions, one application and no deployment. We do not generalise to single-chip Wi-Fi SoCs as a class. The single-core variants of the same family are arguably the more interesting case, since they cannot pin at all.

Model validity of the broker. The access-control arms model broker admission as a per-identity permission on the command topic. They do not model ACL pattern syntax, wildcard subscriptions, bridged or clustered brokers, client-certificate identities, dynamic credential issuance, or detection and revocation after a leak. A deployment differing in any of these may behave differently, and no claim is made

about products we did not test. The ACL semantics the arms assume are no longer only assumed: Section 9.3.1 reproduces every scenario against a real Mosquitto broker, which agreed with the model in all eight cases. What remains unvalidated is everything outside those semantics—the list above—and the broker testbed ran on loopback with a private CA rather than in a deployed network.

Conclusion validity. We report no statistical conclusion from the simulation, precisely because the zero-variance data cannot support one. The formal conclusions are sound within their abstraction, and that abstraction excludes timing. The security conclusions of the campaign follow deductively from the construction and are therefore valid but uninformative; the ablation was added because of this.

Bias. There is no blinding, and the analysis author is the implementation author. For the hardware campaign this matters more than it does in simulation, because expectancy effects can influence setup and outlier handling. The measurement protocol and the analysis script are committed before any data is collected, which is a pre-registration in substance.

13 Limitations and Future Work

The principal limitation is stated plainly: no physical measurement has been performed, and the timing-invariance claim is consequently untested. No latency figure appears in this paper that we have not measured, and none is estimated.

Beyond this: the benign arrival model underlying the sensitivity analysis is synthetic; the per-command CPU cost of tag verification and the energy cost of both the security features and promiscuous-mode monitoring are unmeasured; the access-control arms model broker admission as a per-identity permission, and although Section 9.3.1 confirms those semantics against a real Mosquitto broker, the richer ACL features listed in Section 12 remain unmodelled; no runtime TLS cost is measured anywhere, because the host harness has no TLS implementation and the only figure we report for it is static flash and RAM from the real compiler; and the false-alarm rate of the sensor-fusion modes has not been characterised against real sensors.

Three limitations follow directly from Section 9.3 and are properties of the system rather than of the evaluation. Flash encryption, secure boot and signed over-the-air update are not implemented, so the command key and the broker credentials are both recoverable from a device an attacker can physically reach; this is the assumption on which the signed-command guarantee rests, and it is the weakest one in the design. There is no key rotation or revocation, so recovery from key compromise requires reflashing every holder. And because the replay counter is persisted and monotonic, an adversary holding the key can advance it beyond reach and lock the owner out until the device is reflashed; a bounded-window or timestamp-anchored scheme would trade that failure mode for a dependency on a trusted clock the device does not currently have. We identify these as the design's real exposure and do not claim to have addressed them.

The ordered priorities for future work are therefore: conditions A through C of the hardware protocol, which need only two boards and a probe; extending the broker testbed of Section 9.3.1 to wildcards, bridged brokers and client-certificate identities; enabling flash encryption and secure boot and re-running the compromise arms against a device whose key is no longer extractable; condition D, subject to approval; a passive capture to replace the synthetic arrival model; and replication on a single-core variant of the same family.

14 Conclusion

We treated a consumer intrusion alarm as a safety-critical system and asked whether the one isolation mitigation available on its hardware—core pinning, combined with bounded queue discipline—is adequate when the attacked resource is the chip's own radio. Prior measurement establishes that network

activity reaches a critical task on this hardware despite cross-core separation; what it does not establish is whether the available mitigation suffices, and it has not examined radio-layer load, an adversarial model, or any formal treatment.

We answer the logical half of the question with exhaustive model checking under a network task granted no fairness, with 9,300 seeded adversarial trials and with three million sanitizer-backed fuzzing iterations. We show through ablation that availability, integrity and the safety function are separable properties and that a plausible defence can satisfy one while failing another completely. The same ablation found a defect in our own firmware: a compile-time guard that accepted transport security plus a broker login as sufficient authority for unauthenticated remote disarm. The principle it violates is textbook and the corresponding attack is published, so we claim no new vulnerability; what the experiment shows is that such a guard passes review in silence and costs a safety property rather than merely a security one, and that an ablation with a fair transport baseline is enough to expose it. We removed it. We show through sensitivity analysis that our own initial detector threshold was wrong by a wide margin, and we corrected that too. Extending the ablation to vary the broker's access-control list and the secret the adversary holds then showed that neither broker-side authorization nor application-layer authentication dominates the other—each fails under a compromise the other survives—and produced a negative result we did not anticipate, in which the replay counter that defeats capture-and-replay becomes, under key compromise, a lockout against the legitimate owner. Most of the findings that changed this work were corrections to our own design or to our own measurements, and we report them in that spirit. For the timing half we contribute an analysis and an apparatus rather than a number: a decomposition that removes three of four interference terms by construction and names the one that remains, and a public framework with which an independent laboratory can measure it. We report no latency figure, because we have not measured one.

All claims are scoped to the platform tested: the dual-core Xtensa ESP32 under arduino-esp32 2.0.17 and 3.3.12. The artifact is public so that the results can be reproduced and the open measurement completed, by us or by others.

Declarations

Funding. This research received no external funding.
Conflicts of interest. The author declares no conflict of interest.
Author contributions. MD Humayun: conceptualisation, methodology, software, formal analysis, investigation, data curation, writing (original draft, review and editing), visualisation.
Data availability. All data reported here are generated deterministically from fixed seeds by scripts in the public repository. Aggregate results and the raw per-trial records for the ablation, sensitivity and broker access-control experiments are committed; the campaign's per-trial records regenerate identically by running the provided script. The broker testbed generates throwaway keys and a password file on each run, which are deliberately not committed. No third-party or human-subject data were used.
Code availability. Firmware, formal specifications, experiment harnesses, analysis scripts and figure sources are publicly available under the MIT licence. Continuous integration re-runs the formal verification and a reduced adversarial campaign on every commit.
Ethics. No human or animal subjects were involved, and no third-party network, device or system was attacked or tested. The repository publishes no attack-generation procedure or tooling.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used an AI assistant for drafting assistance, documentation, code review and literature discovery. After using this tool the author reviewed and edited the content as needed and takes full responsibility for the content of the publication. All research claims,

experimental designs, results and conclusions are the author's own; every result reported here was produced by the committed scripts and verified by the author, and every reference was checked against its publisher, proceedings or preprint record.

References

[1] I. Behnke, L. Pirl, L. Thamsen, R. Danicki, A. Polze, and O. Kao, "Interrupting real-time IoT tasks: How bad can it be to connect your critical embedded system to the Internet?," in *Proc. IEEE 39th Int. Performance Computing and Communications Conf. (IPCCC)*, 2020. doi:10.1109/IPCCC50635.2020.9391536.

[2] I. Behnke, P. Wiesner, R. Danicki, and L. Thamsen, "A priority-aware multiqueue NIC design for real-time IoT devices," in *Proc. 37th ACM/SIGAPP Symp. on Applied Computing (SAC)*, 2022. doi:10.1145/3477314.3507165.

[3] I. Behnke, C. Blumschein, R. Danicki, P. Wiesner, L. Thamsen, and O. Kao, "Towards a real-time IoT: Approaches for incoming packet processing in cyber-physical systems," *Journal of Systems Architecture*, vol. 140, art. 102891, 2023. doi:10.1016/j.sysarc.2023.102891.

[4] F. Fischer *et al.*, "Analysis of industrial device architectures for real-time operations under denial of service attacks," arXiv:2007.08885, 2020.

[5] L. Meszlényi, J. Kahle, D. Püllen, S. Kowalewski, S. Katzenbeisser, and A. Kampmann, "Towards timing isolation for mixed-criticality communication in software-defined vehicles," arXiv:2508.13652, 2025.

[6] D. Hoornaert, G. Ghaemi, A. Bastoni, R. Mancuso, M. Caccamo, and G. Corradi, "Mcti: Mixed-criticality task-based isolation," *Real-Time Systems*, vol. 60, no. 2, pp. 328–365, 2024. doi:10.1007/s11241-024-09425-5.

[7] A. Burns and R. I. Davis, "Mixed criticality systems: A review," 14th edn., Dept. of Computer Science, University of York, Feb. 2026.

[8] S. Kremser and K. Graffi, "Evaluating the ESP32-S3 for Wi-Fi penetration testing through the development of Deauther32 and HackHeld32," *Sensors*, vol. 26, no. 11, art. 3287, 2026. doi:10.3390/s26113287.

[9] R. Cayre, D. Cauquil, and A. Francillon, "ESPwn32: Hacking with ESP32 system-on-chips," in *Proc. 17th IEEE Workshop on Offensive Technologies (WOOT)*, 2023.

[10] A. Carbajal and A. Jodeiri Akbarfam, "A software-defined testbed for quantifying deauthentication resilience in modern Wi-Fi networks," arXiv:2602.23513, 2026.

[11] I. Mudraje, K. Vogelgesang, J. Devreker, L. Gerhorst, P. Raffeck, P. Wägemann, and T. Herfet, "Reverse engineering the ESP32-C3 Wi-Fi drivers for static worst-case analysis of intermittently-powered systems," arXiv:2501.17684, 2025. doi:10.1145/3722572.3727926.

[12] M. A. Azhar, M. Azwar, and O. Asroni, "Performance and security analysis of MQTT over TLS on ESP32-based IoT systems," *J. Applied Informatics and Computing*, 2026.

[13] V. Ieropoulos *et al.*, "Empowering IoT security: On-device intrusion detection in resource constrained devices," arXiv:2605.13159, 2026.

[14] M. Krichen, "A survey on formal verification and validation techniques for Internet of Things," *Applied Sciences*, vol. 13, no. 14, art. 8122, 2023. doi:10.3390/app13148122.

[15] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[16] J. Yun, F. Rustamov, J. Kim, and Y. Shin, "Fuzzing of embedded systems: A survey," *ACM Computing Surveys*, vol. 55, no. 7, 2022. doi:10.1145/3538644.

[17] M. Bley, T. Scharnowski, S. Wörner, M. Schloegel, and T. Holz, "Protocol-aware firmware rehosting for effective fuzzing of embedded network stacks," in *Proc. 2025 ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2025. doi:10.1145/3719027.3765125.

[18] D. Tharindu, A. Jayasena, and P. Mishra, "SysFuSS: System-level firmware fuzzing with selective symbolic execution," arXiv:2602.02243, 2026.

[19] W. Alsabbagh, S. Amogbonjaye, C. Kim, and P. Langendörfer, "Pirates of the MQTT: Raiding IIoT systems with a rogue client," in *2024 8th Cyber Security in Networking Conference (CSNet)*, IEEE, 2024, doi:10.1109/CSNET64211.2024.10851733.

[20] H. Zeghida, M. Boulaiche, and R. Chikh, "Security of MQTT protocol: A brief overview," in *Proc. TACC 2024*, CEUR Workshop Proceedings, 2024.

[21] G. Amirkhanova, S. Ismailov, A. Amirkhanov, S. Adilzhanova, M. Zhasuzakova, and S. Chen, "A lightweight, end-to-end encrypted data pipeline for IIoT: An AES-GCM implementation for ESP32, MQTT, and Raspberry Pi," *Information*, vol. 17, no. 1, art. 33, 2026, doi:10.3390/info17010033.

[22] C. Yu, M. Qi, H. Luo, J. Tian, and J. Lu, "MQTT-L: A lightweight end-to-end secure protocol with renewable hash chains for IoT environments," *Journal of King Saud University Computer and Information Sciences*, vol. 38, no. 6, art. 539, 2026, doi:10.1007/s44443-026-00972-4.

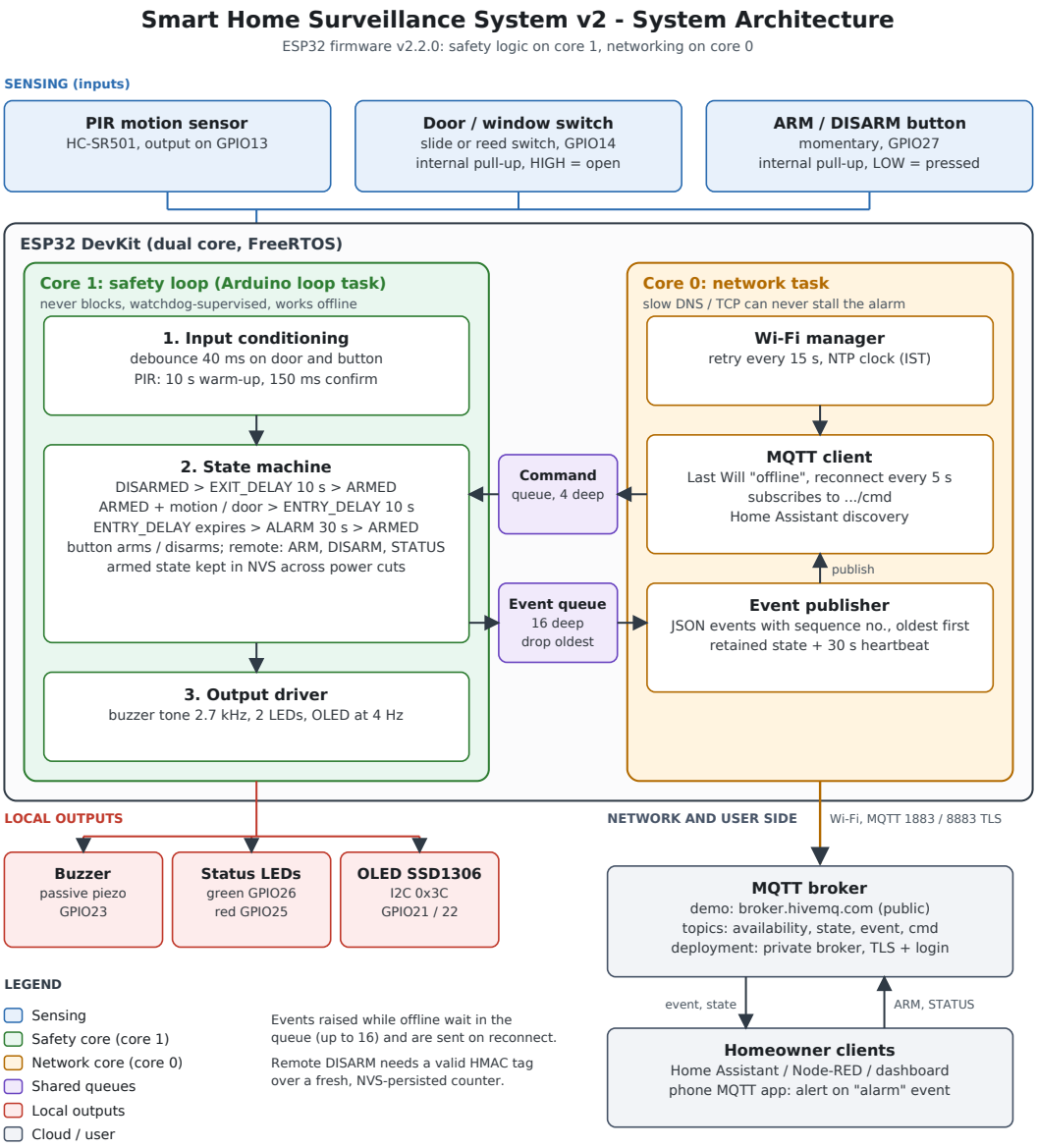


Figure 1: System architecture. The safety domain on core 1 owns the state machine, sensing and actuation; the network domain on core 0 owns Wi-Fi, MQTT and command authentication. The two communicate only through two bounded queues and a status snapshot guarded by a zero-timeout mutex.

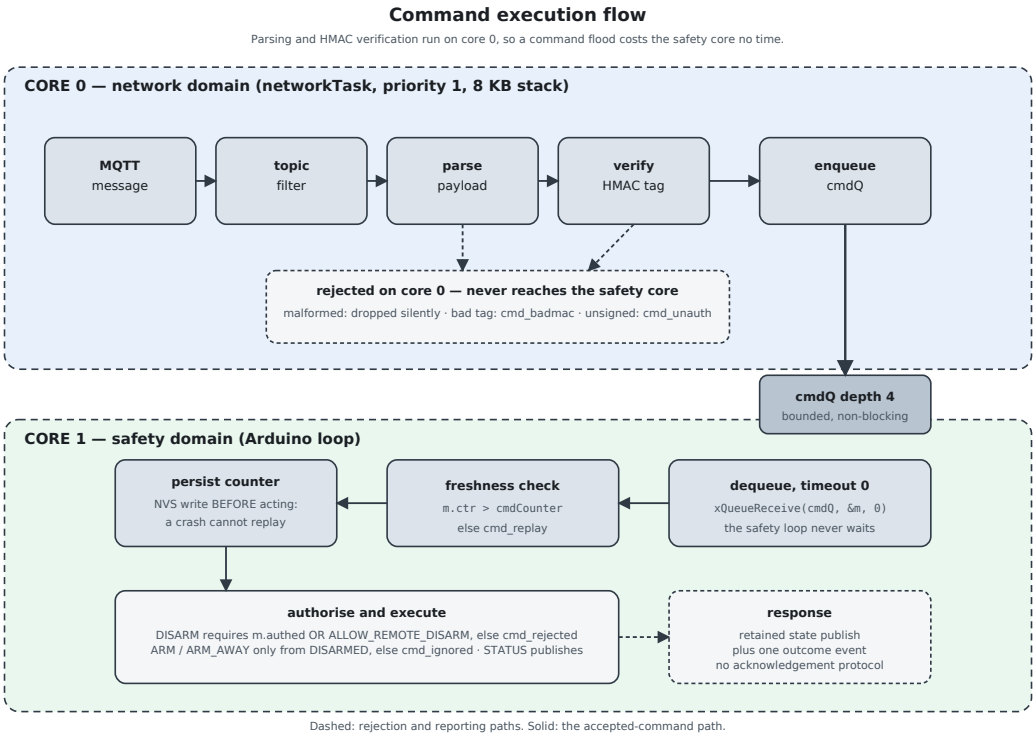


Figure 2: Command execution across the two cores. Parsing and tag verification complete on core 0, so a command flood costs the safety core no time; the replay counter is committed to non-volatile storage before the command is acted upon.

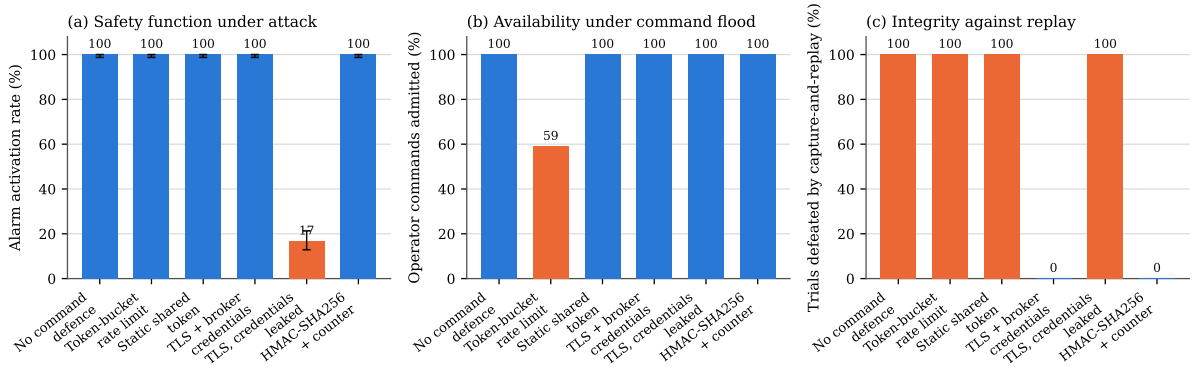


Figure 3: Command-path ablation across the three properties it separates. (a) The safety function holds in every design except the leaked-credential arm; bars give the point estimate and whiskers the 95 % Wilson interval. (b) Only the rate limiter harms operator availability. (c) Capture-and-replay defeats every design except TLS with credentials held and the MAC with a monotonic counter.

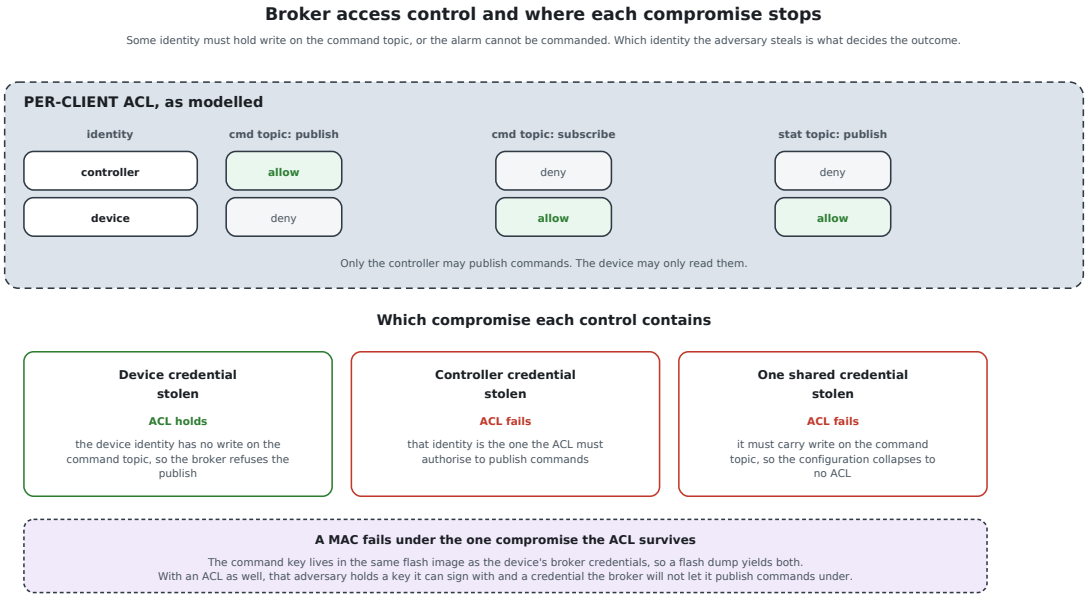


Figure 4: Per-client topic permissions as modelled, and which compromise each control contains. Some identity must hold write on the command topic or the alarm cannot be commanded, so which identity the adversary steals decides the outcome. The MAC fails under the one compromise the ACL survives, because the command key resides in the same flash image as the device's broker credentials.

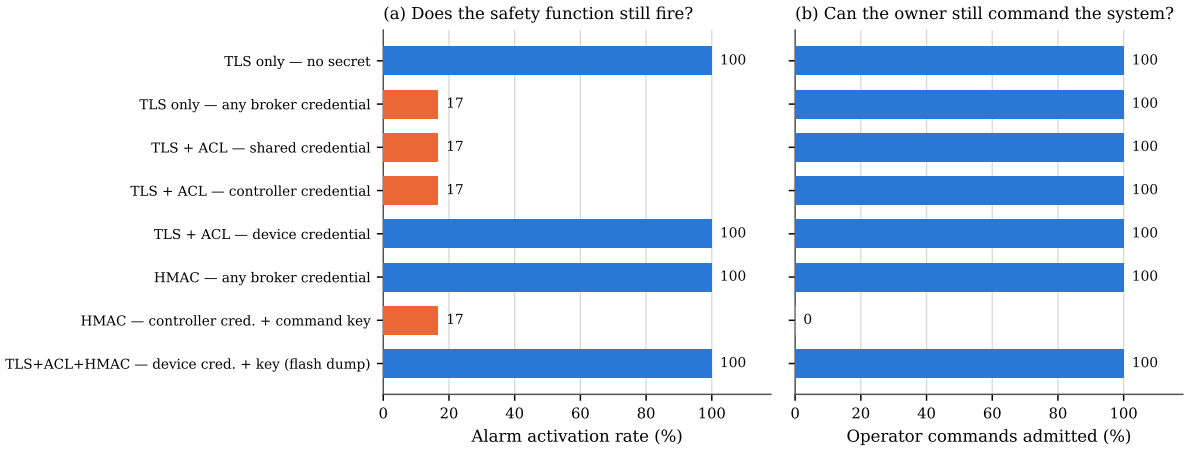


Figure 5: Safety-function activation and operator availability under each combination of controls and compromised secret. No single control survives every row; the layered configuration does.

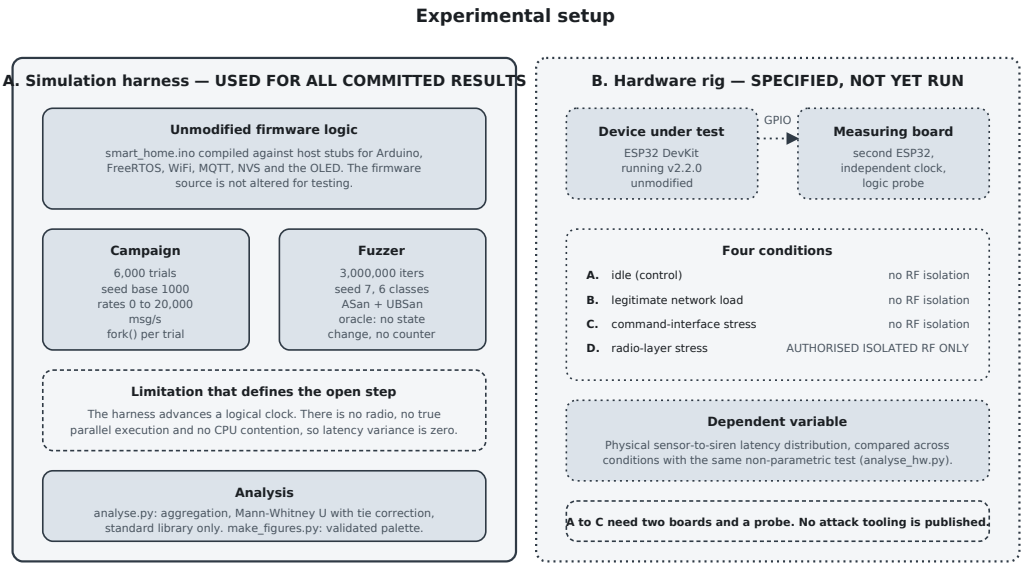


Figure 6: Experimental setup. (A) The simulation harness used for all results reported in this paper. (B) The hardware rig, specified and built but not yet run, in which a second microcontroller serves as an independent measuring instrument across four load conditions.

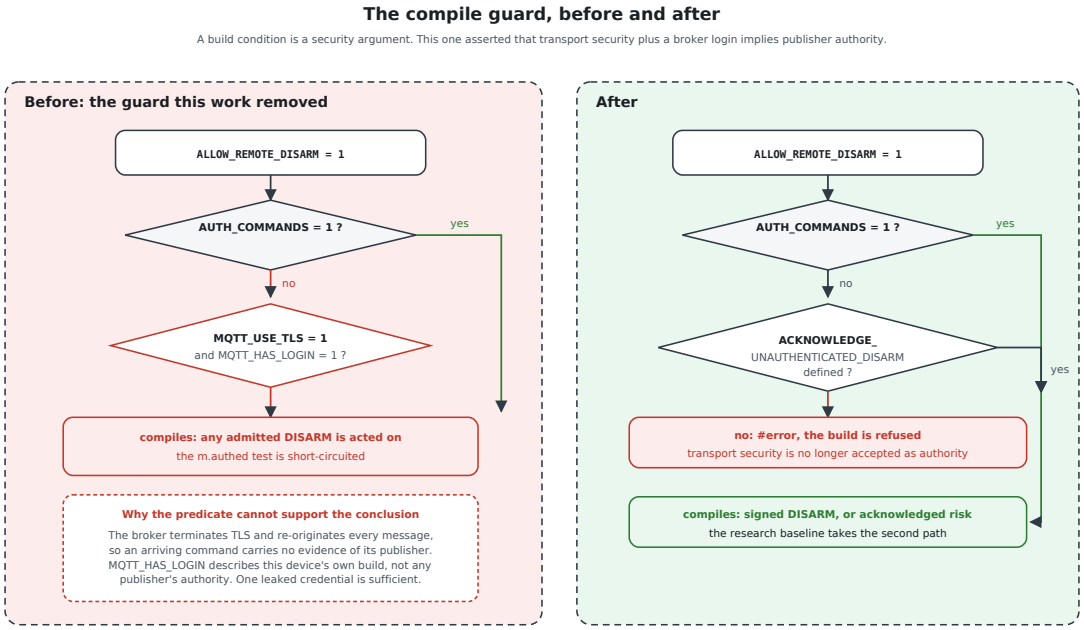


Figure 7: The compile guard before and after. Both branches of the original guard reach the same outcome: a build in which any admitted `DISARM` is acted on. A signed build reaches it legitimately; an unsigned one reaches it through a transport test that cannot establish publisher authority. The repair removes the unsound branch rather than adding a mechanism, and the research baseline reaches the same outcome only through an explicit acknowledgement macro. Transcribed from the preprocessor conditions and from `handleCommands`.

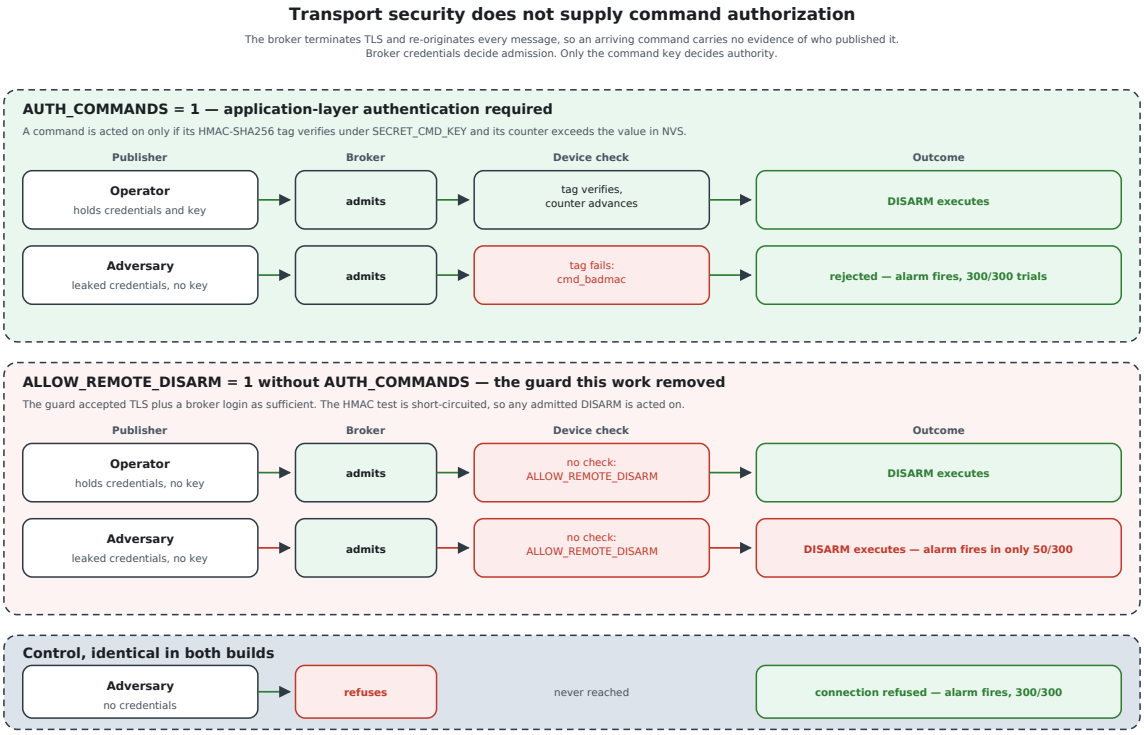


Figure 8: Where broker credentials decide admission and where the command key decides authority. Under `AUTH_COMMANDS` an adversary holding leaked broker credentials but not the command key is rejected and the alarm still fires in all 300 trials; with the removed guard the tag check is short-circuited and the same adversary disarms the system. The control, identical in both builds, shows an adversary without credentials refused at the broker. Every relationship drawn is taken from the implementation; the counts are measured.